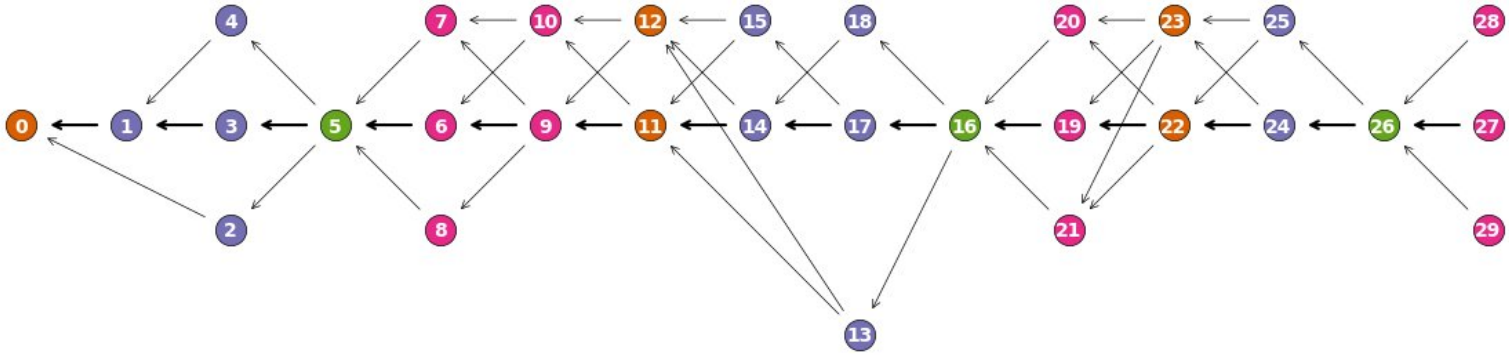


Braidpool

Bob McElrath, Ph.D.

BTC++ Austin
May 8 2025

bob.mcelrath@gmail.com

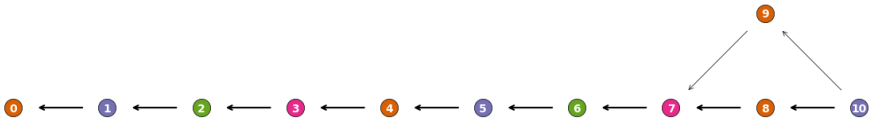
Braidpool will be a *decentralized mining pool* which separates a mining pool into these components:

- Device monitoring
  - Local HTML-based UI to monitor your mine and Braidpool node
- Transaction selection
  - Performed by all Braidpool nodes using a *committed, mempool* and *deterministic block template*
  - Each share (bead) carries 2-5 bitcoin transactions which are added to the committed mempool.
- Variance reduction
  - *Market-Based* through selling shares to market makers
- Payment processing
  - Version 1: PPLNS similar to OCEAN
  - Version 2: *Full Proportional* payout over a 2016-block difficulty adjustment period using financial *options*

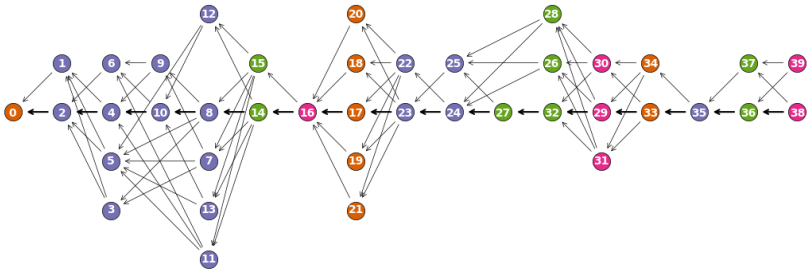
Outline

- The Braid
  - Timeless consensus through cohorts
  - Highest Work Path
  - Simulations
  - Difficulty Adjustment
- Committed Mempool
  - Deterministic Block Templates
  - Metadata Commitments
  - Share Value

What is a Braid?



A thin braid (high difficulty) with a “diamond” structure.



A thick braid (low difficulty) with multi-bead cohorts.

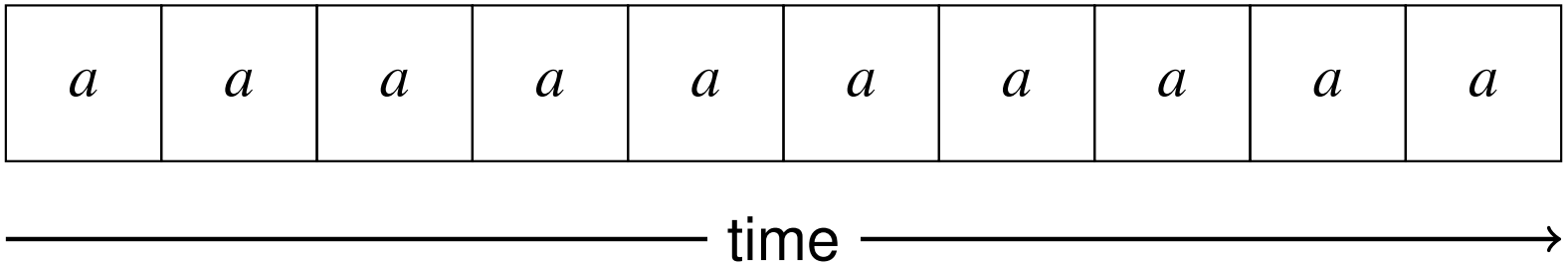
- A Directed Acyclic Graph (DAG) of *beads* (shares)
- Beads must not name ancestors of their parents as parents (“incest”)
- *Cohorts* are sets of beads separated by graph cuts
- Cohorts are a *total order* on the graph and represent consensus points.
- The Highest Work Path (HWP) is indicated in the middle

Braid Statistics (I): Latency a is a "clock"

The arrival time of beads is Poisson distributed with mean $\mu_B = t\lambda x$:

$$P(t,k) = \frac{(t\lambda x)^k e^{-t\lambda x}}{k!} \tag{1}$$

We can model this as a sequence of time intervals of length a



If a time interval a passes with no beads produced, this gives enough time for beads to propagate to nodes and be incorporated into a braid. This *statistically* defines a "cohort". The probability that $k = 0$ beads are produced is $P(t,0) = e^{-a\lambda x}$.

Braid Statistics (II): Expectation Value and Variance

The number of cohorts in a time window also Poisson distributed with mean $\mu_C = P(t, k = 0)\mu_B$. We can then form the expectation value for the number of beads in N_C cohorts:

$$\mathbb{E}\left[\frac{N_B}{N_C}\right] = \mathbb{E}\left[\frac{T_C}{T_B}\right] = \lambda x \mathbb{E}[T_C] = 1 + a\lambda x e^{a\lambda x} \tag{2}$$

where

$$\mathbb{E}[T_C] = \frac{1}{\lambda x} + a e^{a\lambda x} \tag{3}$$

and the variance is

$$\mathrm{Var}\left[\frac{N_B}{N_C}\right] = \frac{\mu_B}{\mu_C^2} + \frac{\mu_B^2}{\mu_C^3} \tag{4}$$

These are the *exact* behavior of a mining network (including bitcoin!) at constant hashrate λ and latency a.

Braid Statistics (III): Target Difficulty

The location of the minimum in T_C is given by

$$\frac{\partial T_C}{\partial x} = 0 \qquad \Longrightarrow \qquad x = x_0 = \frac{2W\left(\frac{1}{2}\right)}{a\lambda} \simeq \frac{0.7035}{a\lambda} \tag{5}$$

where $W(z)$ is the Lambert W function (the solution to $z = We^W$). We can plug this into N_B/N_C to obtain

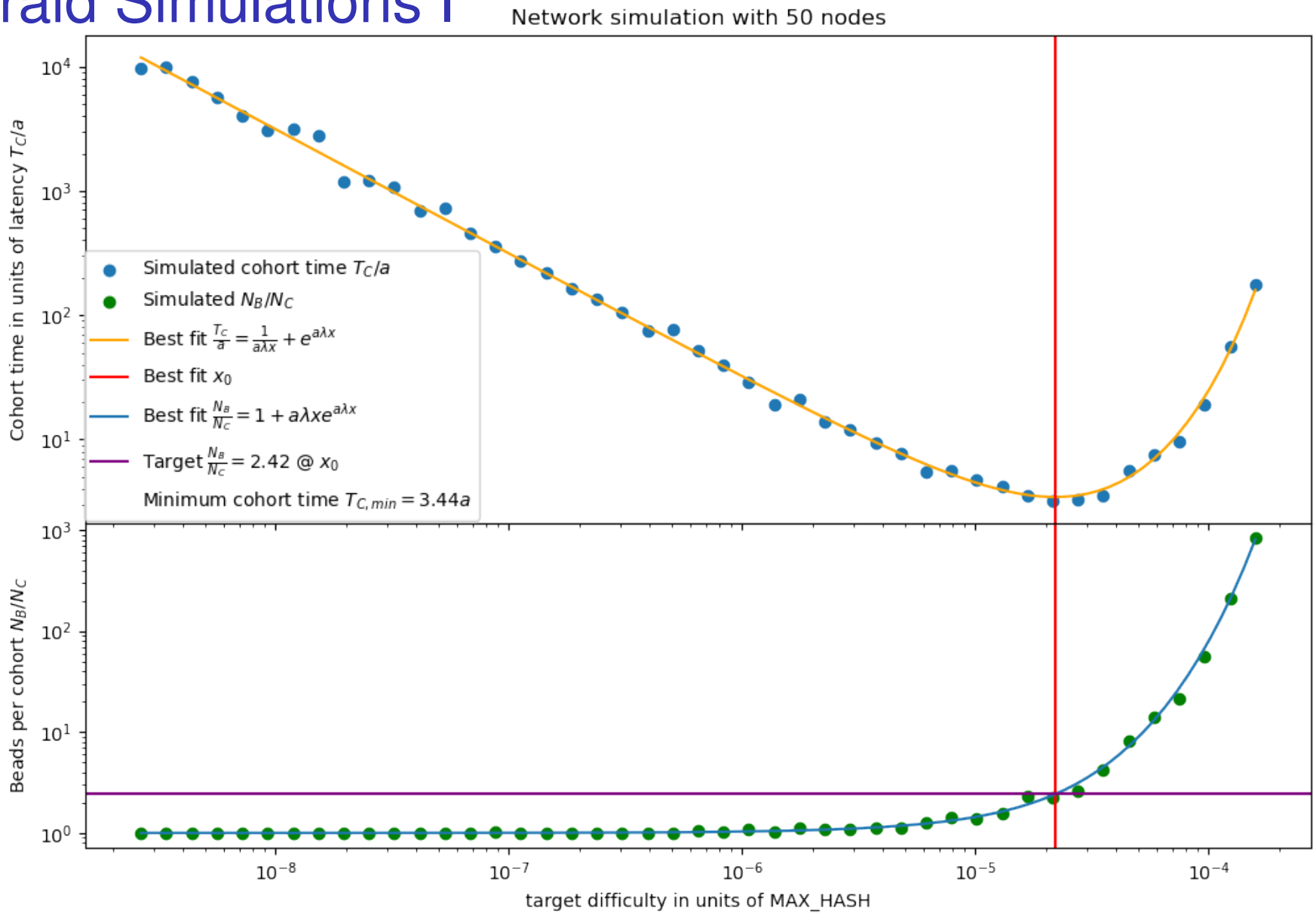
$$\left.\frac{N_B}{N_C}\right|_{x_0} = 1 + \frac{1}{2W(\frac{1}{2})} \simeq 2.4215 \tag{6}$$

at this minimum, the mean cohort time in units of a is

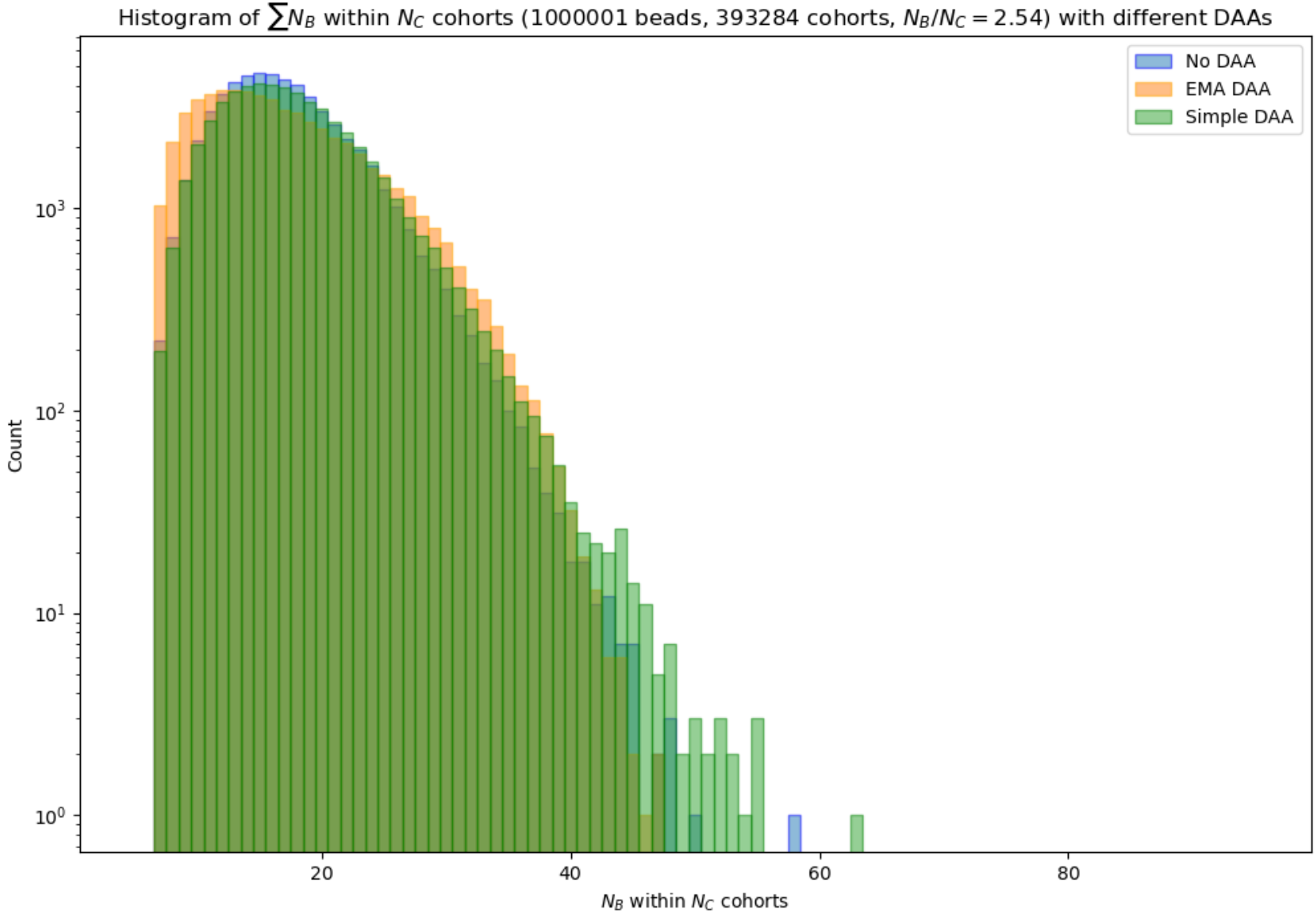
$$\frac{T_{C,min}}{a} = \frac{1}{a\lambda x_0} + e^{a\lambda x_0} = \frac{1}{2W(\frac{1}{2})} + \frac{1}{4W(\frac{1}{2})} \simeq 3.44 \tag{7}$$

This represents having the *most-frequent* graph cuts and is *independent of timestamps*.

Braid Simulations I



Braid Simulations II: Beads within 7 cohorts



Braid Summary

What's going on here?

- We're using measured latency a as a *clock*
- This is extremely resistant to manipulation
- We can't measure the hashrate λ or latency a without timestamps, but we don't need to.
- Instead we measure the *product* $a\lambda$ and adjust to it.
- By targeting the most-frequent graph cuts, we have a parameter free target for our difficulty adjustment

Shares and Weak Blocks

- Beads (shares) are “weak blocks”:they may not meet bitcoin’s difficulty target x_b but meets some lesser difficulty target x.
- A Bead (share) is a *full bitcoin block* including transactions
- Pool members need to know with **certainty** that other beads *could have been* bitcoin blocks had they met the target.
- Beads need to be *small* and quick to validate
- How do you get beads to be small?
  - $\Rightarrow$ by omitting transactions and the coinbase!
- How do you validate something you omitted?
  - $\Rightarrow$ by committing to transactions and *deterministically computing* the template based on previously-seen data

Bead (Share) Structure

```
pub struct Bead {
    pub block_header: BlockHeader,          // Standard Bitcoin block header
    pub committed: CommittedMetadata,       // Committed to in coinbase OP_RETURN
    pub uncommitted: UnCommittedMetadata,   // Not committed to in coinbase
}

pub struct CommittedMetadata {
    pub transactions: Vec<Transaction>,     // 2-5 transactions added to committed mempool
    pub parents: Vec<BeadHash>,             // Parent beads
    pub payout_address: P2P_Address,        // Payout address for this miner
    pub start_time: Time,                   // When the miner started mining this bead
    pub comm_pubkey: PublicKey,             // Pubkey for ECIES encrypted miner communication
    pub miner_ip: AddrV2,                   // Miner IP address
}

pub struct UnCommittedMetadata {
    pub extranonce: i32,                    // Extranonce used to find this solution
    pub broadcast_timestamp: Time,          // When the miner broadcast this solution
    pub parent_bead_timestamps: Vec<Time>,  // Observation when this node saw its parents
    pub signature: Signature                // Signature on the above uncommitted metadata
}
```

- We *compute* the coinbase transaction and block template independently on each node in order to validate a bead

Different types of miners

Imagine 4 different types of miners

- Profit maximizing miner
- Empty block miner
- Monkey JPEG miner taking fees out of band
- BitAxe on a Raspberry Pi behind a dial-up line

These miners are *not the same* and will earn different revenues. The social contract between pool members is that “I’ll pay you if you pay me”. That implies that they have the same or similar ability to make revenue and the only difference is their hashrate. *We can make them the same through the Committed Mempool*

What's a Committed Mempool?

- A restricted, release-tagged instance of bitcoind: no network access communicates over IPC, used only for its mempool
- Goal is to *prevent* block template outsourcing
- Every bead has an associated committed mempool implied by its parents.
- All transactions not yet mined in all ancestors are already in its committed mempool.
- Add the 2-5 transactions committed by this bead.
- Compute block template with any *deterministic* algorithm.

Advantages of a Committed Mempool

- ALL Transactions are fully validated
  - $\Rightarrow$ All nodes know with confidence that all *other* beads are not going to withhold blocks or generate orphans
- ALL Nodes broadcast blocks as soon as they see the Bead
  - $\Rightarrow$ Impact of low bandwith or high latency nodes is removed
- Transaction selection is a legal liability and no one wants to do it. Ethereum has only MEVil and Compliant miners.
  - $\Rightarrow$ Solve this problem by making *everyone* do it
- Beads are *small* $\sim 20\text{kb}$ so don't cause huge latency problems
- Most of the data for block validation is already known and re-usable.

Transaction Selection

- Large OP_RETURN must be stored forever by all nodes. (you can't compute txids without it)
- Trying to "filter" invites censorship attempts by regulators. Thankfully it's almost entirely useless.
- Braidpool mempool and can have different rules for its mempool.
- Parasitic assets and DEXes are the real threat.
  - ⇒ Bets that can be placed on secondary markets can exceed the block reward and incentivize games
  - ⇒ Order books on Bitcoin incentivize transaction ordering games
  - ⇒ Should we consider transaction order randomization?
- I'm okay with 140 bytes OP_RETURN for ZKP type uses

Share Value

The fundamental unit of reward in Bitcoin is sha256d hashes computed. This is “work”. But because of difficulty targeting, we need to allow shares to have different amounts of work.

Version 1 will use a full-proportional version of PPLNS.

- Expire old shares (window based on time and/or or work)
- Beads with very large cohorts can’t pay everyone: large cohorts are an orphan risk.
  - $\Rightarrow$ Sort beads based on:
    - simple sum of descendant work
    - simple sum of ancestor work
    - hash value (“luck”)
  - $\Rightarrow$ For $N_B \gtrsim 20$, lowest work beads won’t be rewarded
- We’re researching something better. PPLNS sucks.

Comparison to other projects

- HydraPool (P2PoolV2) (jungly)
  - Does not validate transactions: miners can't know if other hashers on the pool are paying them.
  - Difficulty adjustment is dependent on timestamps and easily manipulable
- Hashpool (Cashu eCash) (vnprc)
  - Uses eCash tokens to pay miners
  - Would be great for paying small miners, but must be custodial
  - A similar custodial pool running atop Braidpool paying over Lightning would be most welcome
- OCEAN/TIDES
  - Blocks sent by creator and spot checked

Miner-Selected Difficulty and Sub-Pools

- As long as the miners chosen difficulty is $x_b < x < x_0$ there's nothing wrong with allowing miners to choose their own, since we know how to add work.
- Braidpool will choose your difficulty for you such that all miners have the same *variance*. This cannot be enforced by consensus.
- Miners too small to achieve constant variance will be grouped into a *sub-pool*, which:
  - has outputs in the parent pool as the outputs managed by the sub-pool
  - has an independent Braid from the parent pool

Financial Instruments

- We do not directly create any instant-payout (Lightning-like) method, instead we envision hashers trading shares for BTC
- Professional market makers and risk-taking trading firms would buy shares on centralized exchanges or via atomic swaps. (This is outside the scope of Braidpool, but enabled by it)
- With both shares (per epoch) and BTC as instruments, private hashrate derivative contracts can be created.
- Pools are currently doing this risk management function, and might continue to do so atop braidpool, along with their other services (management, monitoring, financing)

General Timeline

- Braidpool V1: summer 2025
  - Node software (Bob McElrath with Summer of Bitcoin students: Abdullah Azeem, Aritra Majumder, Ansh Sharma, Mohd Zaid)
  - Dashboard software (Bob McElrath with Summer of Bitcoin students: Abhishek, David Borg, Priya Rani)
- Braidpool V2
  - Transactions (sending/trading shares)
  - Latency optimization
  - Sub-Pools
  - Options-based payout mechanism

Summary/Conclusions

- All the elements are present to construct a truly trustless decentralized mining pool (Shares, Braid consensus, PPLNS)
- Demo!